**Universidade de Brasília — CIC0236 Teleinformática e Redes 2 — 2026/2**

# RDT-UnB: transferência confiável de dados sobre UDP com Stop-and-Wait — Relatório da Entrega 1

Grupo `grupo07`: Lucas Adeodato de Oliveira (231013645), Lurian Correia Lima (222005395), Atila Sousa Fernandes (231013510)

## 1 Introdução

O UDP não garante entrega, ordem nem integridade dos dados [2]. Neste trabalho implementamos, sobre socket UDP nativo, o cliente do protocolo RDT-UnB v1.0, que acrescenta confiabilidade à transferência de arquivos a partir dos mecanismos descritos nas Seções 3.4 e 3.5 de Kurose & Ross [1]: números de sequência, reconhecimento (ACK), temporizador e retransmissão, e verificação de integridade por *checksum* MD5 [3].

Esta primeira entrega implementa o protocolo mais simples, o *Stop-and-Wait* (SAW): o cliente solicita um segmento, espera o `DATA` correspondente, confirma com `ACK` e só então pede o próximo. Os três arquivos (*small*, *medium* e *large*) foram recebidos com MD5 correto e o throughput medido ficou entre 99,4% e 99,9% do máximo teórico da Equação 1, em torno de 140–156 kbps, valor determinado quase por completo pelo RTT de aproximadamente 26–30 ms.

## 2 Cliente implementado

### 2.1 Protocolo e fluxo

O servidor é *pull-based*: nunca envia dados sem ser solicitado. O cliente, executado como

```
python3 cliente.py --server 137.131.178.229 --group grupo07 --file small
```

segue o fluxo:

1. **Handshake:** envia `HELLO|grupo=<id>|segment_size=512|file=<arquivo>` e recebe `OK` com `file_size`, `checksum`, `seed`, `total_segments` e o `segment_size` negociado.
2. **Transferência:** para cada $n = 0, \ldots, N-1$, envia `REQ|seq=`$n$, aguarda `DATA|seq=`$n$`|...` até o *timeout* (retransmitindo o `REQ` se ele expirar) e responde `ACK|seq=`$n$.
3. **Encerramento:** calcula o MD5 do arquivo remontado, envia `FIN|checksum=<md5>` e recebe `DONE|ok=1` (íntegro) ou `DONE|ok=0`.

Decisões de implementação:

- O *timeout* é fixo em 1000 ms (`--timeout`). O estimador de RTT (EWMA) só alimenta a coluna `rtt_estimated_ms` nesta entrega.
- No `DATA`, só as três primeiras barras `|` separam campos; o restante é o *payload* binário, que pode conter o byte `|`. O tamanho do payload é conferido contra o campo `total`.
- `DATA` duplicado, fora de ordem, malformado ou vindo de outro endereço é descartado; o cliente continua esperando o `seq` corrente até o *timeout*.
- Por timeout, o `REQ` é reenviado até 10 vezes por segmento; o `HELLO` é enviado até 5 vezes e o `FIN` é retransmitido até 5 vezes. Esgotadas as tentativas, a sessão falha, mas o CSV dos segmentos já recebidos é gravado.

### 2.2 Máquina de estados

A Figura 1 mostra a máquina de estados do cliente.

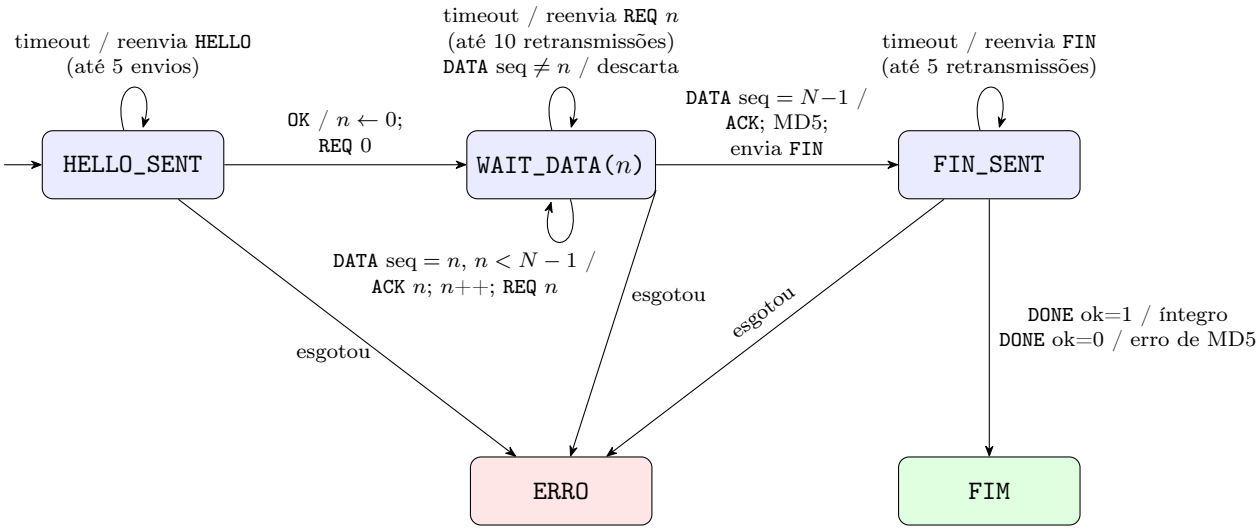



Figura 1: Máquina de estados do cliente Stop-and-Wait. Rótulos no formato *evento / ação*; $N$ é o `total_segments` informado no `OK`. Ao chegar em `FIM` ou `ERRO`, o CSV de métricas é gravado. Com `DONE` ok=0 o cliente apenas reporta a falha de integridade; não refaz o download.

### 2.3 Métricas registradas

Ao fim de cada sessão o cliente grava `metrics_<grupo>_saw_<timestamp>.csv`, com uma linha por segmento e as colunas `segment`, `timestamp`, `size_bytes`, `rtt_ms`, `rtt_estimated_ms`, `timeout_ms`, `throughput_kbps`, `retransmissions`, `timeout_occurred`, `window_size` (sempre 1) e `protocol` (`saw`). O RTT de um segmento é o tempo entre o envio do `REQ` e a chegada do `DATA` correspondente, e o throughput instantâneo é `size_bytes` $\times$ 8/`elapsed_ms`, em que `elapsed_ms` é o tempo desde o primeiro `REQ` do segmento. Há uma sutileza no código: com retransmissão o RTT é medido a partir do *último* `REQ` (não se sabe a qual `REQ` o `DATA` responde, como no algoritmo de Karn), então o `rtt_ms` não inclui o tempo gasto nos timeouts. Já o throughput instantâneo usa o tempo desde o *primeiro* `REQ`, de modo que um timeout aparece como queda de vazão. Em `rtt_ms` aparece o RTT "limpo" e, no throughput, o custo total do segmento.

## 3 Resultados

Os testes foram feitos com `segment_size` de 512 bytes e timeout fixo de 1000 ms, contra o servidor `137.131.178.229`, em 15/10/2026 entre 10h22 e 10h41, sem perda simulada. Os CSVs foram registrados e utilizados para a geração dos gráficos.

### 3.1 Integridade dos arquivos recebidos

A Tabela 1 confirma, para os três arquivos, que o MD5 calculado pelo cliente coincide com o informado no `OK` e que o servidor respondeu `DONE|ok=1` (captura do terminal na Figura 2). Os valores também conferem com `md5sum` sobre os arquivos salvos.

Tabela 1: Checksum MD5 dos arquivos recebidos.

| Arquivo | Tamanho (bytes) | Segmentos | MD5 do arquivo recebido | DONE |
|---|---|---|---|---|
| small | 1 048 576 | 2 048 | `317d6890df9867ccda199afe6a362aba` | ok=1 |
| medium | 5 242 880 | 10 240 | `69e62b40de4415285dffbecf2b494644` | ok=1 |
| large | 10 485 760 | 20 480 | `22473c90e995d0eff6ff8006d17de17c` | ok=1 |

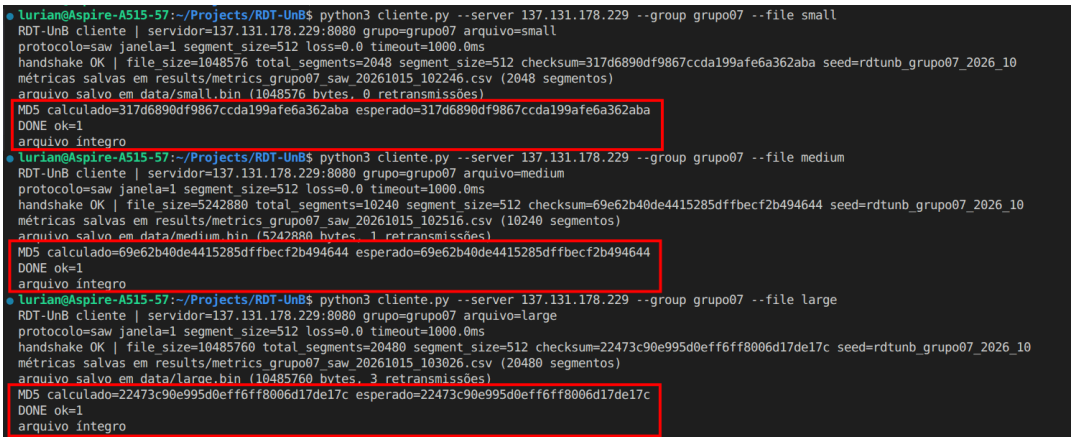



Figura 2: Saída do terminal para os três arquivos; o MD5 calculado coincide com o esperado e o servidor responde `DONE|ok=1`.

## 3.2 Throughput e RTT por segmento

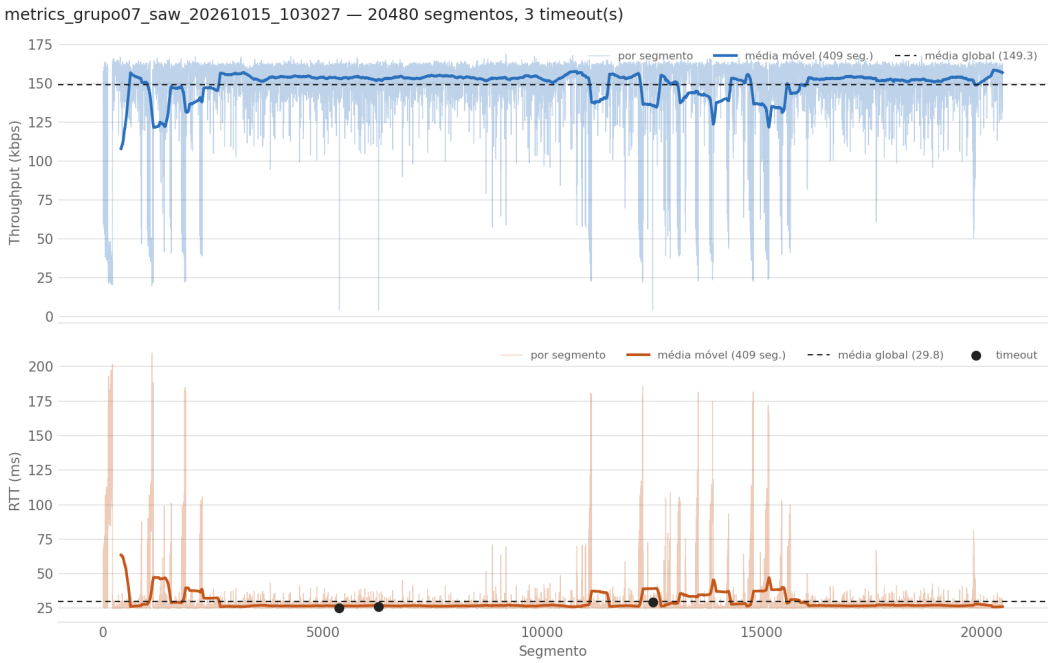



Figura 3: Throughput instantâneo (topo) e RTT (base) por segmento na transferência do arquivo *large* (20 480 segmentos). Os pontos pretos marcam os 3 segmentos com timeout e retransmissão; a curva grossa é a média móvel de 409 segmentos.

No *large*, o RTT típico fica perto do mínimo, de 24–26 ms (mediana de 26,05 ms, média de 29,8 ms), com picos isolados de 100 a 210 ms, concentrados em alguns trechos (início, perto dos segmentos 11 000 e entre 12 000 e 15 000). O throughput é a imagem espelhada do RTT: fica em torno de

150–160 kbps quando o RTT está perto de 26 ms e cai no mesmo segmento em que o RTT sobe (um RTT de 200 ms dá apenas 20 kbps). Os 3 timeouts (segmentos 5379, 6277 e 12 513) não coincidem com picos de RTT, são perdas de pacote. Em cada um deles o segmento custou cerca de 1,03 s, mas o gráfico mostra um RTT normal porque o `rtt_ms` é medido a partir do último `REQ`. A Figura 4 mostra o throughput em janelas de 10 s para os três arquivos: as quedas aparecem como episódios de alguns segundos com RTT alto, também nos arquivos *small* e *medium*.

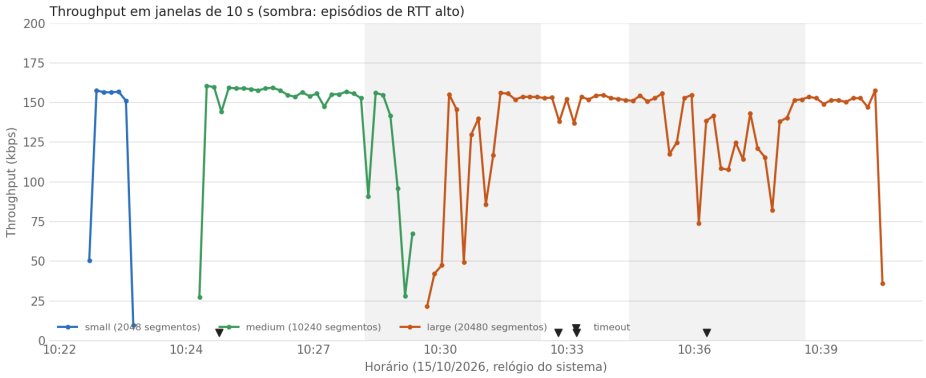



Figura 4: Throughput médio em janelas de 10 s nas três execuções; as regiões sombreadas são episódios de RTT alto e os triângulos, timeouts.

### 3.3 Throughput teórico e medido

O throughput máximo teórico do SAW é

$$\text{Throughput}_{\text{máx}} = \frac{\texttt{segment_size} \times 8}{\text{RTT}_{\text{médio}}}, \tag{1}$$

e o throughput medido é o total de bytes recebidos multiplicado por 8 e dividido pelo tempo total da transferência (do primeiro `REQ` ao último `DATA`). Nessa definição, os timeouts entram no tempo total. O RTT médio é a média da coluna `rtt_ms` do CSV.

Tabela 2: Throughput teórico (Eq. 1) e medido para os três arquivos.

| Arquivo | RTT médio (ms) | Teórico (kbps) | Medido (kbps) | Medido/ teórico | Retrans-missões | Tempo total (s) |
|---|---|---|---|---|---|---|
| small | 26,24 | 156,1 | 155,9 | 99,9 | 0 | 53,8 |
| medium | 28,27 | 144,9 | 144,2 | 99,5 | 1 | 290,7 |
| large | 29,84 | 137,3 | 136,4 | 99,4 | 3 | 614,8 |

## 4 Análise

### (a) Por que o throughput do SAW é limitado pelo RTT?

No SAW há no máximo um segmento em trânsito: o cliente só pede o segmento $n+1$ depois de receber o $n$. Cada segmento custa, portanto, pelo menos um RTT, e o número de segmentos por segundo é no máximo 1/RTT, qualquer que seja a capacidade do enlace. Com 512 bytes por segmento e RTT de $\approx 26$ ms, o limite é $\approx 38$ segmentos/s, ou $\approx 156$ kbps. Para a utilização do emissor $U = (L/R)/(\text{RTT} + L/R)$ [1], usando como exemplo um enlace de $R = 100$ Mbps (valor assumido, não medido), $L/R = 4096/10^8 \approx 0{,}04$ ms e $U \approx 0{,}04/26{,}3 \approx 0{,}16\%$: o canal fica ocioso durante praticamente todo o tempo. Aumentar a largura de banda não ajuda; só reduzir o RTT ou ter mais de um segmento em trânsito (janela deslizante) aumenta a vazão. Os dados confirmam isso: o throughput por segmento acompanha 1/RTT (Figura 3), com $\approx 160$ kbps para RTT de 25 ms e

≈ 20 kbps nos segmentos com RTT de 200 ms, e o throughput médio muda de arquivo para arquivo na mesma proporção do RTT médio (156,1 contra 137,3 kbps quando o RTT médio sobe de 26,2 para 29,8 ms).

### (b) Throughput teórico máximo × medido

Para o *large*, a Equação 1 dá $512 \times 8/29{,}84 \approx 137{,}3$ kbps, e o medido foi $10\,485\,760 \times 8/614{,}8 \approx 136{,}4$ kbps, 99,4% do teórico. No *small* o RTT médio foi 26,24 ms (teórico 156,1 kbps, medido 155,9 kbps, 99,9%) e no *medium* 28,27 ms (144,9 contra 144,2 kbps, 99,5%). Os resultados do SAW ficam muito próximos do teórico porque a fórmula já usa o RTT médio medido e o cliente não faz nada além de esperar o RTT.

A diferença restante vem dos timeouts. Como o `rtt_ms` não inclui o tempo de timeout, o teórico não os enxerga, mas o tempo total de transferência sim: cada timeout custa o valor inteiro de 1000 ms. No *large*, 3 timeouts somam 3 s de 614,8 s (≈ 0,5%), que explica quase toda a diferença de 0,6%; no *medium*, 1 s de 290,7 s (≈ 0,3%); no *small*, que não teve timeouts, sobram 0,1%, do processamento do cliente (*parsing*, montagem do ACK, registro das métricas), que ocorre fora da janela do RTT. A queda do throughput com o tamanho do arquivo se deve ao RTT médio mais alto nos arquivos maiores: os picos de RTT de 100–200 ms (Figura 4) puxam a média para cima e quanto mais longa a execução, mais episódios desses entram na média; e a cada timeout o tempo parado equivale a $\approx 1000/26 \approx 38$ segmentos. Isso mostra que, com timeout fixo de 1000 ms, bastam poucas perdas para custar vazão, e motiva o timeout adaptativo da próxima entrega.

## 5 Conclusão

O cliente recebeu os três arquivos íntegros (MD5 conferido e `DONE|ok=1`), com 0, 1 e 3 retransmissões. O throughput medido ficou entre 99,4% e 99,9% do teórico, de 136–156 kbps, bem abaixo da capacidade do enlace, pois o SAW fica limitado a um segmento por RTT. Isso motiva os protocolos de janela deslizante (GBN e SR) da Entrega 2.

## 6 Referências bibliográficas


[1] J. F. Kurose e K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken: Pearson, 2021.

[2] J. Postel, "User Datagram Protocol," RFC 768, IETF, 1980. Disponível em: https://www.rfc-editor.org/rfc/rfc768.

[3] R. Rivest, "The MD5 Message-Digest Algorithm," RFC 1321, IETF, 1992. Disponível em: https://www.rfc-editor.org/rfc/rfc1321.